

Energy Performance

Technical Report

By: Alisar Sanwar

1 Problem statement, and loading and preparing the data

1.1 Problem statement

The research center wants to improve the energy performance for its buildings, through advanced analytical methods. So, data have been collected including the factors that may affect the energy performance in Winter (heating load) and Summer (cooling load).

In the data collected dataset there are 8 different factors (features) that may affect energy performance.

1. Surface Area: the total external area (without counting windows area)
2. Relative The ratio of the surface area of a reference building to the actual surface area of the building having the same volume ($A_{\text{reference}}/A_{\text{actual}}$), bounded between 0 and 1.
3. Roof Area: the total surface of a building's roof
4. Wall Area: the walls total Area
5. Orientation: building orientation can only be (North, South, East, West) it is a categorical data that should be encoded
6. Overall Height: the total height of a building (have a big effect on surface to volume ratio) tall buildings need more energy load in both Summer and winter season
7. Glazing Area: The total area of window glazing
8. Glazing Area Distribution: A categorical variable indicating how windows are distributed across facades (unglazed, uniformly, or concentrated on one side). It must be numerically encoded for analysis.

The labels:

1. Heating Load: the amount of energy that needed for every building to keep the building warm in the Winter
2. Cooling Load: the amount of energy that needed for every building to keep the building at a cool temperature during Summer

In order to analyse the data, we should ensure that the data is accurate, this can be achieved through the data cleaning process we ensure that the data do not contain duplicate records, wrong values or null values in the data set and if there are null values they should be filled by machine learning model. Also, all data features should be numerical features in order to to implement a numerical operations on the features. If there are categorical features in the data, they should be encoded.

After that the data will be ready to be analysed and drawing conclusions from it. Visualization figures will be a great helper in this stage.

1.2 Loading the dataset

The data was saved in CSV file with .csv extension and is saved in the same path of the code file, so a read_csv function from pandas library was used in order to read the data and it is stored in df variable (with dataframe variable type)

```
# reading the file
# read.csv will read csv file and transform it into dataframe
df = pd.read_csv ("energy_heating_cooling.csv")
```

The column names in the data were unclear (x1, x2, ...) as they did not reflect the physical parameters stored within them, so they needed to be changed to more meaningful names.

```
# now we will change column name to be more clear
# first we will make a dictionary and connect the current column name with the new one

col_new_names = {
    'X1' : 'Surface_Area',
    'X2' : 'Relative_Compactness',
    'X3' : 'Roof_Area',
    'X4' : 'Wall_Area',
    'X5' : 'Orientation',
    'X6' : 'Overall_Height',
    'X7' : 'Glazing_Area',
    'X8' : 'Glazing_Area_Distribution',
    'Y1' : 'Heating_Load',
    'Y2' : 'Cooling_Load'
}

# now we will use rename function in pandas
# inplace = the new changes will happen on current df
df.rename (columns= col_new_names , inplace = True )

# checking the new column names
print (df.head ())
```

I have used a dictionary to change the columns names rather than list, because in dictionary the interpreter will search to the column in the data that has the same name as the key in the dictionary and change the column name to be same as that key value, and if a column name does not match any dictionary key the interpreter will not change the column name.

While in list it will change the columns value depending on order and if the developer enters the columns' names in wrong order the columns' names will be wrong as well. So, I found it more useful to use dictionary rather than list.

To rename the columns a rename function was used, and (.inplace) has been chosen as True which means I want these changes to be done on the original data frame and not on a copy from it

X1	X2	X3	X4	X5	X6	X7	X8	Y1
514.5	100.00	110.25	294.0	North	7.0	0.0	No glazing	15.55
514.5	100.00	110.25	294.0	East	7.0	0.0	No glazing	NaN
514.5	100.00	110.25	294.0	South	7.0	0.0	No glazing	15.55
514.5	100.00	110.25	294.0	West	7.0	0.0	No glazing	15.55
563.5	0.90	122.50	318.5	North	7.0	0.0	No glazing	20.84
563.5	0.90	122.50	318.5	East	7.0	0.0	No glazing	21.46
563.5	0.90	122.50	318.5	South	7.0	0.0	No glazing	20.71
563.5	0.90	122.50	318.5	West	7.0	0.0	No glazing	19.68
588.0	0.86	147.00	294.0	North	7.0	0.0	No glazing	19.50
588.0	0.86	147.00	294.0	East	7.0	0.0	No glazing	19.95

The columns before changing their names.

Surface_Area	Relative_Compactness	Roof_Area	Wall_Area	Orientatic
514.5	100.0	110.25	294.0	North
514.5	100.0	110.25	294.0	East
514.5	100.0	110.25	294.0	South
514.5	100.0	110.25	294.0	West
563.5	0.9	122.50	318.5	North
Overall_Height	Glazing_Area	Glazing_Area_Distribution	Heating_Load	
7.0	0.0	No glazing	15.5	
7.0	0.0	No glazing	NaN	
7.0	0.0	No glazing	15.5	
7.0	0.0	No glazing	15.5	
7.0	0.0	No glazing	20.8	
NaN				

The columns after renaming.

To discover the data a lot of functions has been used:

```
# exploration of the dataset

print ('\n Data Shape : ', df.shape)
print ('\n Data Types : \n', df.dtypes )
# df.info should viwe the number of rows and columns & data types & non null values & uses
print ('\n Data Information : ' )
df.info()
# shows the count, min, max, avrg ...
print ('\n Numerical Summary Statistics: ', df.describe())
print ('\n the Data Size : ', df.size)
```

The functions:

1. shape: display the (rows number, columns number)

```
Data Shape : (798, 10)
```

2. dtypes: display the type for each column

```
Data Types :
Surface_Area          float64
Relative_Compactness  float64
Roof_Area             float64
Wall_Area             float64
Orientation           str
Overall_Height        float64
Glazing_Area          float64
Glazing_Area_Distribution str
Heating_Load          float64
Cooling_Load          float64
dtype: object
```

3. info (): display the number of columns and rows the type of each column, the number of non-null values in each column and the memory usage.

```
Data Information :
<class 'pandas.DataFrame'>
RangeIndex: 798 entries, 0 to 797
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Surface_Area                        798 non-null   float64
1   Relative_Compactness                798 non-null   float64
2   Roof_Area                          798 non-null   float64
3   Wall_Area                          798 non-null   float64
4   Orientation                        798 non-null   str
5   Overall_Height                     798 non-null   float64
6   Glazing_Area                       798 non-null   float64
7   Glazing_Area_Distribution           798 non-null   str
8   Heating_Load                       755 non-null   float64
9   Cooling_Load                       755 non-null   float64
dtypes: float64(8), str(2)
memory usage: 62.5 KB
```

4. describe (): it shows the Numerical Summary Statistics for each column (mean, min, max...)

```
Numerical Summary Statistics:
count    Surface_Area  ...  Cooling_Load
mean      671.355263  ...    24.698927
std       87.980942   ...    9.540650
min       514.500000   ...   10.900000
25%       612.500000   ...   15.640000
50%       661.500000   ...   22.720000
75%       735.000000   ...   33.235000
max       808.500000   ...   48.030000
```

5. size: The number of cells in the data frame (number of columns * number of rows)

```
the Data Size : 7980
```

Key points concluded:

- There are 798 rows and 10 columns with 7980 cells in the data frame
- All columns' data types are float except Orientation column and glazing area distribution column their data type is string
- Heating load and Cooling load columns are the only columns that contain null values (need to be imputed using a machine learning model)

1.3 Preparing and pre-processing the dataset

After discovering the data, a cleaning stage will be the next step, from the discovering stage.

The First step is removing the duplicated records if exist:

```
show duplicated recordes :
   Surface_Area  ... Cooling_Load
768      759.5  ...      13.88
769      784.0  ...      20.29
770      588.0  ...      35.99
771      612.5  ...      30.17
772      514.5  ...      25.88
773      686.0  ...      13.72
774      514.5  ...      25.84
775      759.5  ...      18.14
776      710.5  ...      14.58
777      637.0  ...      45.97
778      612.5  ...      28.20
779      612.5  ...      29.93
780      563.5  ...      33.82
781      661.5  ...      36.85
782      661.5  ...      39.85
783      808.5  ...      14.15
784      637.0  ...      36.85
785      637.0  ...      39.41
786      514.5  ...      33.76
787      759.5  ...      13.88
```

After ensuring that the data contain duplicated records, deleting these records to raise the analysing accuracy process will be the next step.

```
# cleaning the data

# get rid of any duplicated recordes
# df.duplicated return a df indexes with a boolean series
print ("\nshow duplicated recordes : \n", df[df.duplicated()])
print ("\n Number of duplicated recordes : ", df.duplicated().sum())

# now to remove the duplicated recordes

df.drop_duplicates(inplace= True)

# check that all duplicates had been dropped
print ("\nThe number of remaining duplicates ",df.duplicated().sum())
# if it is = zero all the duplicates recordes was  dropped successfully
```

After removing all duplicated records (30 records) we should ensure that no remaining duplicated records exist.

```
Number of duplicated recordes : 30
The number of remaining duplicates 0
```

The second step of cleaning is ensuring the data integrity.

There are 2 columns in the data with string values type, for example Orientation column which should contain only 4 unique values, 'North', 'South', 'East' and 'West'.

To check for the Unique values for Orientation column .unique() function is used

```
# Standardizing the format
# the orientation column should only contain 4 unique values
# to check
print('\n\nUnique values for Orientation column : ', df['Orientation'].unique())

print('The number of the current vlaues in the orientaion column is : ', len (df['Orientation'].unique()))
# strip is a function delete the extra spaces before and after
```

```
Unique values for Orientation column :  <StringArray>
['North', 'East', 'South', 'West', ' East', 'west', 'south', ' North']
Length: 8, dtype: str
The number of the current vlaues in the orientaion column is : 8
```

Python found 8 different unique values in Orientation column while there should be only 4 values. Python considered 'west' is a different value than 'West' and considered ' North' is a different value than 'North'. To get rid of this problem the data should be cleaned by standardizing the data structure. In order to do that both .capitalize() function (a function which make all letters lower case except the first one it makes it upper case) and .strip() function (remove all extra spaces before and after the word) should be used.

```
# strip is a fuction delete the extra spaces before and after
# capitalize make all the letters small except the first make it capitale letter
df['Orientation'] =df['Orientation'].str.strip().str.capitalize()
```

```
Check the Orientaion values after cleaning :  <StringArray>
['North', 'East', 'South', 'West']
Length: 4, dtype: str
```

Now there are four unique values in the Orientation column as it should be.

Then we should ensure that Glazing Area Distribution are accurate

```
# check the unique vales in Glazing_Area_Distribution
print ('\n\nCheck the Glazing_Area_Distribution values : ', df['Glazing_Area_Distribution'].unique())
# the values in this column does not need cleaning
```

```
Check the Glazing_Area_Distribution values : <StringArray>
[ 'No glazing',      'Uniform', 'North glazing', 'East glazi
'South glazing', 'West glazing']
Length: 6, dtype: str
```

We found that Glazing_Area_Distribution column does not require cleaning.

Relative Compactness column values are the result of dividing the current surface area for a building/ the perfect surface area for a building if both of them own the same volume. However, all these values should be between the 0-1 and any value out of this range will be considered as wrong value. In the data set a group of wrong values in that column (above 100) so, each value above 1 will be changed to be one.

```
# now cleaning Relative_Compactness column data
# the values in this column should be between 0-1

df.loc[df['Relative_Compactness'] > 1, 'Relative_Compactness'] = 1

# to check if now all the Relative_Compactness are now in the range
print ('\n The minimum value in Relative_Compactness column is : ', df['Relative_Compactness'].min())

print ('\n The maximum value in Relative_Compactness column is : ', df['Relative_Compactness'].max())
```

```
The minimum value in Relative_Compactness column is :    0.62

The maximum value in Relative_Compactness column is :    1.0
```

The minimum and maximum values in the column now are in (0-1) range means this cleaning step was successful.

The third step is to check the number of null values in each column using .isna() function which return a boolean data frame (if the value is null it will be True and if not it will be false) then after using .sum() function it will count the number of True values in each column.

```
# now checking the number of null values
# isna return a dataframe of boolean values and isna.sum return the number of null values in each column
print ("\n Missing values for each column\n", df.isna().sum())
print ("\nthe totla number for missng values : ", df.isna().sum().sum())
```

```

Missing values for each column
Surface_Area          0
Relative_Compactness  0
Roof_Area             0
Wall_Area             0
Orientation           0
Overall_Height        0
Glazing_Area          0
Glazing_Area_Distribution  0
Heating_Load         41
Cooling_Load          41
dtype: int64

the total number for missing values : 82

```

Each one of Heating load and Cooling load columns contain 41 null values.

The method to be adopted for handling these values is imputing them using a machine learning that will be built in the next section, which is more effective than deleting the records that contain null values as row deletion leads to a substantial loss of valuable data, and imputing these null values by the average is not chosen because it lacks accuracy. As a result, building a machine learning will be the best choice.

In order to implement a numerical operation on the columns values, all the column data type should be numerical. So, any string values will get encoded.

```

# Categorical Encoding

# we can not make a numerical operation on string values
# so the string values in Orientation and Glazing_Area_Distribution should be converted

# we will make a copy because the original dataframe will be needed in visualization purposes
df_copy = df.copy()

# we will use label encoding
from sklearn.preprocessing import LabelEncoder
# the original encoded will stay at the data
df['Orientation_Encoded'] = LabelEncoder().fit_transform(df['Orientation'])
df['Glazing_Area_Distribution_Encoded'] = LabelEncoder().fit_transform(df['Glazing_Area_Distribution'])

df_encoded = df
# check on the changes that happen after encoding

print(df_encoded.head(3))

```

The encoded type that has been implemented in the code is label encoding, which means each unique value will get consecutive numerical values starting with 0 and ending with number of unique values - 1.

The columns after encoding will be saved in new columns in the data frame

Orientation_Encoded	Glazing_Area_Distribution_Encoded
1	1
0	1
2	1

2 Advanced data processing

The previous section shows that the target columns contained missing values, and the reasons behind using imputing these missing values by machine learning methods were explained. Using mean replacement results to flatten distribution variance which affects the data accuracy, and using deleting records sacrifices critical observations. As a result, a machine learning approach using Linear Regression Imputation was applied in both Heating Load column and Cooling Load column.

```
# Heating Load
# # to train the model we will need the rows that have not null value
obs_heating = df_encoded['Heating_Load'].notna()
```

```
# now we will do the same steps for Cooling_Load

obs_c = df_encoded['Cooling_Load'].notna()
```

To build a machine learning model, this model should be trained in a correct and known records. So, the first step is identifying these records to be used in the training the model.

```
_cols_h = ['Surface_Area', 'Relative_Compactness', 'Roof_Area', 'Wall_Area',
           'Overall_Height', 'Glazing_Area', 'Orientation_Encoded', 'Glazing_Area_Distribution_Encoded']
```

```
predictor_cols_c = ['Surface_Area', 'Relative_Compactness', 'Roof_Area', 'Wall_Area',
                    'Overall_Height', 'Glazing_Area', 'Orientation_Encoded', 'Glazing_Area_Distribution_Encoded']
# find the correlation
```

Then all columns that might be chosen to train the model should be identified, these columns should not contain null values.

```
# find the correlation
# this is very important line which is responsible for measuring the correlations between the predictor_cols and the target
corr_h = df_encoded.loc[obs_heating, predictor_cols_h + ['Heating_Load']].corr()['Heating_Load'].drop('Heating_Load')
# take the top 2 correlation columns and save their values in a list
top2_features_h = corr_h.abs().sort_values(ascending=False).index[0:2].tolist()

print("\n\n\n Top correlated features with Heating_Load:", top2_features_h)

# train the model
```

```
# find the correlation
corr_c = df_encoded.loc[obs_c, predictor_cols_c + ['Cooling_Load']].corr()['Cooling_Load'].drop('Cooling_Load')

top2_features_c = corr_c.abs().sort_values(ascending=False).index[0:2].tolist()

print("\n\n\n Top correlated features with Cooling_Load:", top2_features_c)
```

Then the correlation between all the columns and the heating load and cooling load columns will be measured.

Note** `.drop('Heating_Load')` and `.drop('Cooling_Load')` functions were implemented because `.corr()` function calculate the correlation between the column and itself which will always be one, and it won't be useful while implementing the machine learning model. So, `.drop()` function is used to drop that column.

Not all columns will be chosen to train a model, but only the highest two correlation columns with the target column, and these two columns are 1. Overall Height 2. Roof Area.

The reason behind taking only the highest two and all the features is preventing the noise that might happen if low correlation columns has been used in building the model, for example the correlation between the building Orientation and the Heating load and cooling load is very low so using the Orientation column to build the model will make noise and won't make the model more accurate.

And we did not only use the highest correlation features (Overall Height) because will make the model weak and will decrease the accuracy, we will take the highest 2 features correlation.

```
Top correlated features with Heating_Load: ['Overall_Height', 'Roof_Area']
```

```
Top correlated features with Cooling_Load: ['Overall_Height', 'Roof_Area']
```

Training the model:

```
# train the LinearRegression model
x_train_h = df_encoded.loc[obs_heating , top2_features_h].values
y_train_h = df_encoded.loc[obs_heating , ['Heating_Load']].values

model_h = LinearRegression()

model_h.fit(x_train_h , y_train_h)
```

```
# train the model
x_train_c = df_encoded.loc[obs_c , top2_features_c].values
y_train_c = df_encoded.loc[obs_c , ['Cooling_Load']].values

model_c = LinearRegression()

model_c.fit(x_train_c , y_train_c)
```

By identifying the features (Overall Height, Roof Area) → X

And identifying the label (Heating Load or Cooling Load) → Y

Then building a Linear Regression object and the `.fit()` method was called to train the model on the feature matrix and target vector. The Linear Regression object will extract Mathematical equation in this shape

$$Y = a * X_1 + b * X_2 + c \text{ (where a, b, and c are constant parameters)}$$

```

# predict the values
# now making the model predicting the vlaues
# saving the missing values in a variable
missing_h = df_encoded['Heating_Load'].isna()
# checking if there are any missing values
if missing_h.any():
    fulling_h = model_h.predict(df_encoded.loc[missing_h , top2_features_h].values)
    # adding the prediction values for both df_encoded and df_copy
    df_encoded.loc[missing_h , 'Heating_Load'] = fulling_h
    df_copy.loc[missing_h , 'Heating_Load'] = fulling_h
# printing the results
print('\nthe number of missing values in Heating_Load after fulling the missing values : ', df_encoded['Heating_Load'].isna().sum())

print(df_encoded['Heating_Load'])

```

```

# predict the values
missing_c = df_encoded['Cooling_Load'].isna()
if missing_c.any():
    fulling_c = model_c.predict(df_encoded.loc[missing_c , top2_features_c].values)
    df_encoded.loc[missing_c , 'Cooling_Load'] = fulling_c
    df_copy.loc[missing_c , 'Cooling_Load'] = fulling_c

print('\nthe number of missing values in Cooling_Load after fulling the missing values : ', df_encoded['Cooling_Load'].isna().sum())

print(df_encoded['Cooling_Load'])

print("\n--- Final Dataset Missing Values:" , df_encoded.isna().sum().sum())

```

After training the model on the existing values, the next step will be predicting the null values by passing the records that contain the null values and using the .predict() function with the model that we have trained. And the model will return the predicted values to impute the missing entries in the data frame.

After that we should ensure there are no missing values in the target columns by .isna().sum() functions

```

the number of missing values in Heating_Load after fulling the missing values : 0
0      15.550000
1      31.100068
2      15.550000
3      15.550000
4      20.840000
...
763     17.880000
764     16.540000
765     16.440000
766     16.480000
767     16.640000
Name: Heating_Load, Length: 768, dtype: float64

```

```

the number of missing values in Cooling_Load after fulling the missing values : 0
0      21.33000
1      32.43024
2      21.33000
3      21.33000
4      28.28000
...
763     21.40000
764     16.88000
765     17.11000
766     16.61000
767     16.03000
Name: Cooling_Load, Length: 768, dtype: float64

```

```
--- Final Dataset Missing Values: 0
```

Notice that the data set does not contain any missing values which indicates that imputing missing values is done successfully.

3 Data Visualization

In this section both Matplotlib and Seaborn have been used to generate visualization figures.

Matplotlib library is Low-level visualization library; means it give the programmer precise and detailed control over every element within the drawing window.

Seaborn library is built above Matplotlib library, and it is High-level visualization library, and it is designed to deal smoothly with Panda's data frame. Its shortcut the long complex codes in one line.

1)Heatmap (Correlation Analysis):

its target is to measure the strength and direction of the linear relationships between the data set columns using correlation.

```
# first a heat map to show the correlation between columns

# deciding the figure size
# the active figure now is heat map
plt.figure(figsize=(12,10 ))

# finding the correlation
corr_matrix = df_encoded.corr(numeric_only=True)

sns.heatmap(corr_matrix , annot= True , cmap= 'coolwarm', fmt = '.2f' )

plt.title('Correlation Heatmap between the features and labels ')
plt.show()
```

Used functions:

** plt is the shortcut for Matplotlib library

- .figure() → generating a figure
- .figsize(numeric number (for width) , numeric number (for height))
- .corr → a function used to calculate the correlation between the columns, and numeric_only = True → because df_encoded contains columns with string values (Orientation , Glazing Area Distribution) and correlation can be calculated only by numeric numbers so this option tell the function to ignore any values that not numeric.
- sns.heatmap() → a seaborn function that generates the heatmap.
 - annot= True → write every correlation value in its square this will ease the readability of the map.
 - camp = 'coolwarm' → (colour map) this refers to the color scale used for visualization: cool blue represents a strong negative correlation (-1), while

warm red represents a strong positive correlation (+1), with neutral colors (grey and white) indicating values close to zero (no correlation).

- `fmt = '.2f'` → decide the numeric values format (.2f means floating point with 2 decimal places)
- `plt.title()` → decide the title
- `plt.show()` → display the figure

2. Density Plot (KDE Distribution)

Its target is to display the probability distribution form for the continuous labels to help in analysing it and facilitating the comparison of their locations and distribution.

```
# Creating the density plot
# second visualization
plt.figure(figsize=(8,6))

# the first density for heating
sns.kdeplot(data = df_encoded['Heating_Load'], fill = True , color= 'red' , label = 'heating load' )
# the second density for cooling
sns.kdeplot(data= df_encoded['Cooling_Load'], fill= True , color= 'blue' , label = 'Cooling Load')

plt.title('Density Plot for the heating and cooling distrubution ')
plt.xlabel('enargy load')
plt.ylabel('Density(Data ratio and concentration)')
plt.legend()
plt.show()
```

Used functions:

- `.kdeplot()` → a function in Seaborn used to generate density plot.
 - `data=` → what is the data you want this figure to display (Heating load column)
 - `fill = True` → fill the shape with a colour
 - `color = red` → use a red colour to display the drawing
 - `label = 'heating load'` → label this drawing with heating load
 - ** `.kdeplot()` function was used twice once with Heating load column and once with Cooling load column, to compare between two results drawing
 - `data=` → what is the data you want this figure to display (Cooling load column)
 - `fill = True` → fill the shape with a colour
 - `color = blue` → use a blue colour to display the drawing
 - `label = 'cooling load'` → label this drawing with cooling load
 - `x.label()` → the x axis label in the figure
 - `y.label()` → the y axis label in the figure
 - `plt.legend()` → Renders the visual key mapping line colors to corresponding energy labels
- ** use labels in the drawing to facilitate knowing what each drawing represents.

3.Box Plot:

Its target to compare the distribution, median, and spread of heating load across different building heights, and to detect any potential outliers .

```
# The box plot
# shows the relation between the hight and the heating load

plt.figure(figsize= (8,6))

sns.boxplot(x= 'Overall_Height' , y= 'Heating_Load' , data= df_encoded)

plt.title('Box Plot (the relation between heating load the building hight )')
plt.xlabel('Building overall height')
plt.ylabel('Heating load')

plt.show()
```

Used functions:

- sns.boxplot() → generating a box plot drawing.
 - X = 'Overall_Height' determine the variable in x axis which is responsible for dividing the data to groups according to its values.
 - Y= 'heating load' determine the variable in y axis which is responsible for drawing the box plot depending on the heating load for every group.
 - data=df_encoded → the source of data
- ** Note: the other used functions have explained in the previous figures.

4. Scatter Plot with Hue (Multivariate Analysis)

"A scatter plot reveals relationships or association between two variables. Such relationships manifest themselves by any non-random structure in the plot... A scatter plot is a plot of the values of Y versus the corresponding values of X:

- Vertical axis: variable Y—usually the response variable.
- Horizontal axis: variable X—usually some variable we suspect may be related to the response. (National Institute of Standards and Technology, 2012).

It is used to represent a three-dimensional relationship between the Heating load, surface area and glazing area.

```
# Scatter Plot , (we will use hue)
# using hue will help us compairing between 3 different variables rather than 2
# it will find the connection between both (surface Area and Glazing Area ) and Heating load

plt.figure(figsize=(8,6))

sns.scatterplot(data= df_encoded , x= 'Surface_Area' , y= 'Heating_Load' , hue= 'Glazing_Area')

plt.title('Scatter Plot')

plt.xlabel('Surface Area')
plt.ylabel('Heating load')

plt.show()
```

Used functions:

- `sns.scatterplot ()` → use to generate a scatterplot drawing
 - `data =` → the source of the data (data frame)
 - `x=` → the variable in x axis
 - `y=` → the variable in y axis
 - `hue =` → the third variable will be represented by the dot colours

5.Bar Plot (Categorical Aggregation)

Purpose: calculate and display the mean of using the energy according to the four Orientations.

```
# the fifth visualization figure
# Bar Plot
# finding the impact of building orientaion on Heating load
plt.figure(figsize=(8,6))
sns.barplot(data=df_copy, x='Orientation', y='Heating_Load')
plt.title('the impact of building orientaion on Heating load ')
plt.xlabel('Building Orientation')
plt.ylabel('Mean heating load')
plt.show()
```

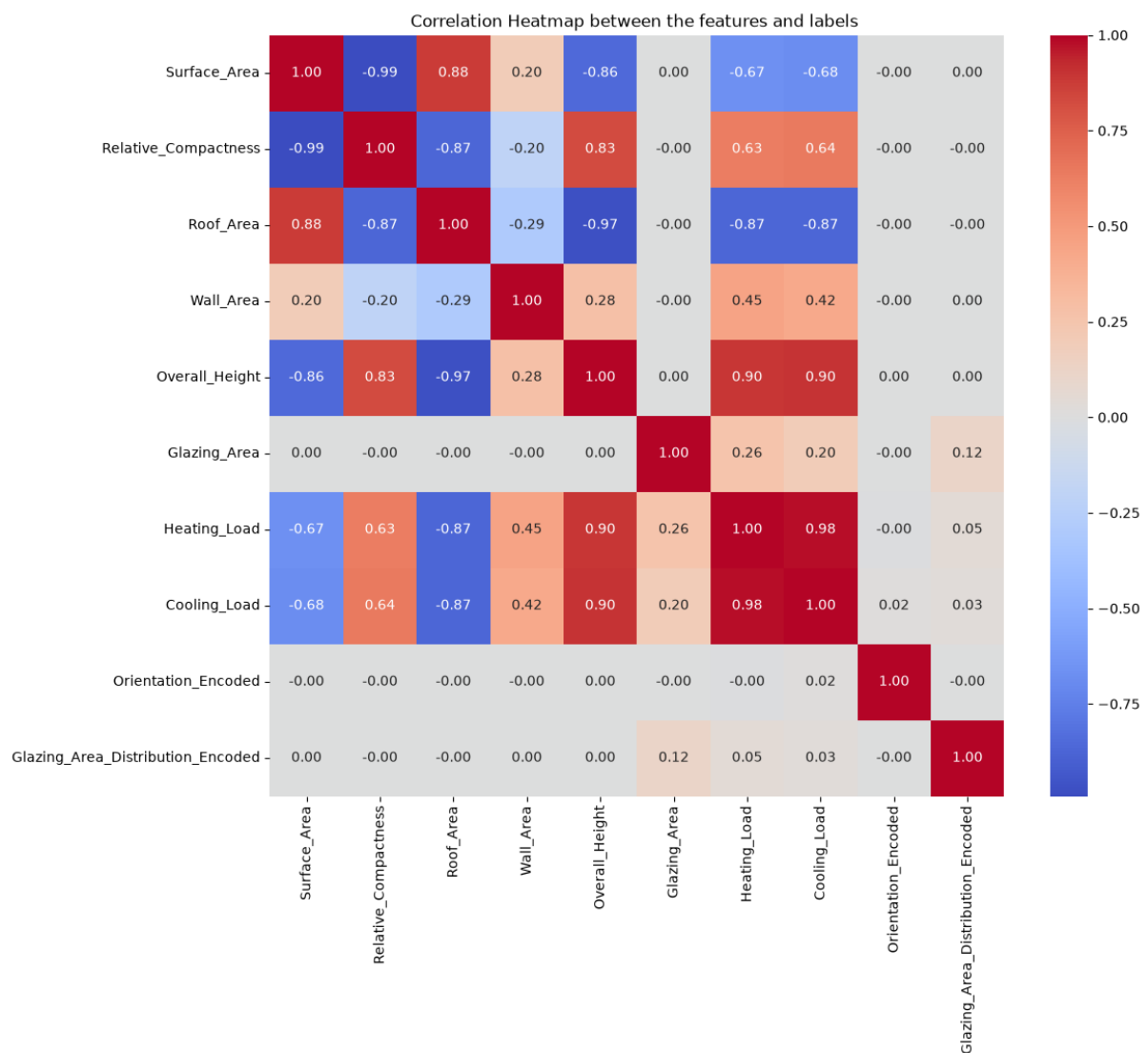
Used functions:

- `sns.barplot ()` → use to generate a bar plot drawing each orientation will be represented in different bar
 - `data =` → the source of the data (data frame)
 - `x=` → the variable in x axis (responsible of dividing the data to groups)
 - `y=` → the variable in y axis (the mean of heating load for each group)

4 Visualizations analysis and Evaluation

4.1 Visualizations analysis

1) Heatmap figure:



It is a 2D matrix, in both x and y axes it shows the columns' names in the data frame and each cell contains a Pearson correlation coefficient between the 2 columns (the correlation number is a value between (-1 to 1) and each cell is coloured with red, blue, grey or in between.

The cells present strong positive correlations, their numeric values be around 1 and their colour is strong red according to coolwarm colour palette.

The cells present strong negative correlations, their numeric value be around -1 and their colour is strong blue according to coolwarm colour palette

Finally, the cells that presents weak correlation or no correlation their numeric value be around 0 and are displayed in neutral grey.

The diagonal cells represent the correlation of each variable with itself, so they are always strong red cells with 1 value correlation.

The heatmap shows a strong correlation between heating load and cooling load = .98 which means the building that need a lot of energy to warm it in Winter will mostly also need a lot of energy to cool it in Summer.

Also, the heatmap shows a strong positive correlation between Overall Height ($r = 0.90$), indicating that taller buildings require significantly higher energy loads in both Summer and Winter seasons.

The correlation between Roof Area and Heating and Cooling load is strong negative, means the building with a big roof area need less energy load than the building with small Roof area.

Buildings with larger roof areas in this dataset correspond to single-story, low-height structures (3.5 m), which have significantly lower overall heating and cooling demands.

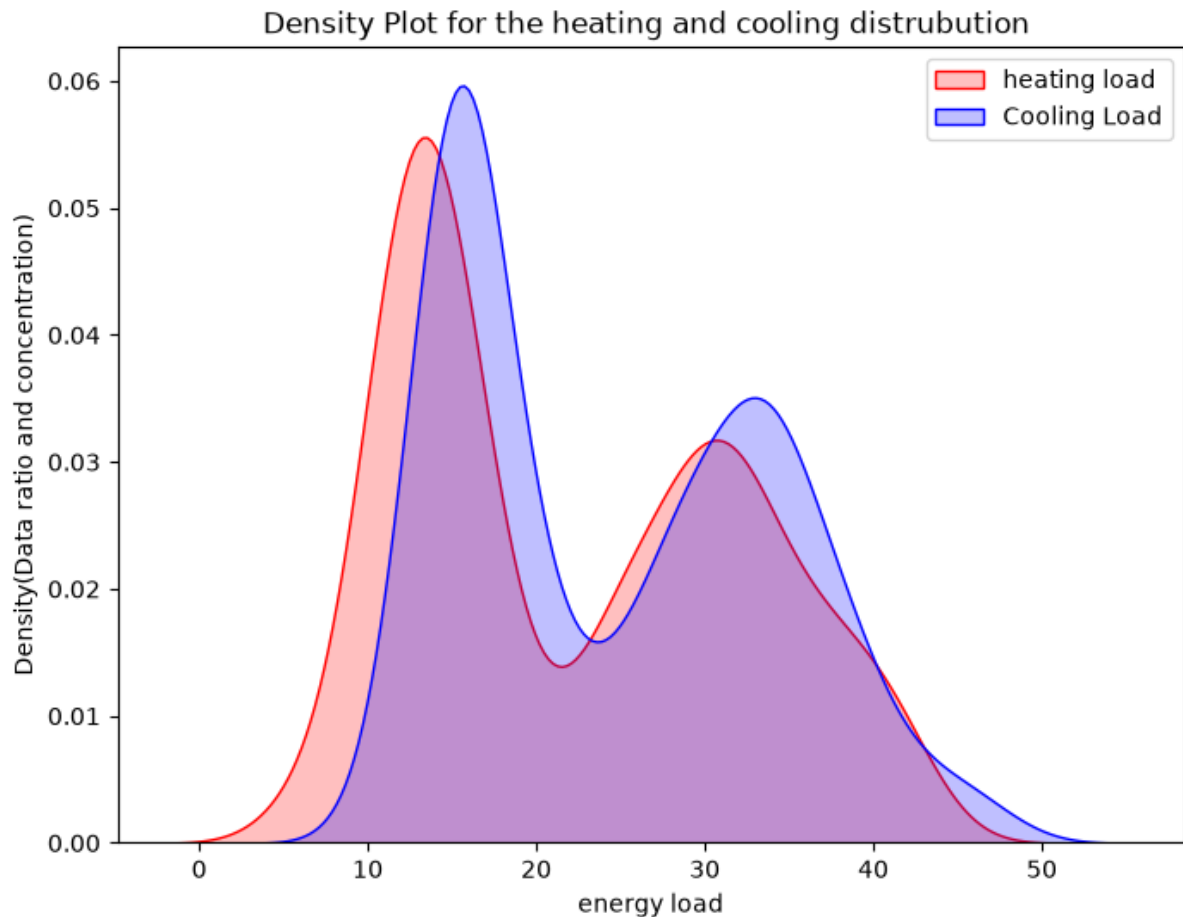
In addition, there is strong negative correlation between the surface Area and Relative compactness, this aligns with the concept of each of them. From a physical standpoint, the greater the building's surface area, the smaller its compactness.

The greater the total height of the building, the more its volume increase which is reflected in the heatmap as a strong positive correlation ($r = 0.83$) between the Overall feature and the relative compactness.

Most of the correlation cells in Orientation and Glazing Area Distribution is grey which means these two features do not affect other variables in the data frame.

Conclusion: The building's structural dimensions (height and roof area) are the primary determinants of energy loss, whereas geographic orientation exhibits no significant linear relationship with annual thermal loads.

2. Density Plot



This figure displays the Kernel Density Estimation (KDE) distributions for both Heating Load and Cooling Load, and it shows the big similarity between the two.

The red drawing represents the Heating load values, and the blue drawing represents the Cooling load values. And the purple represents the intersection of the two graphs.

The total area of Each graph in density plot is 1.

And it is clear that the energy needed in cool season is little more than the energy needed is heating season, as the cooling curve is distinctly shifted to the right towards higher energy values to the right.

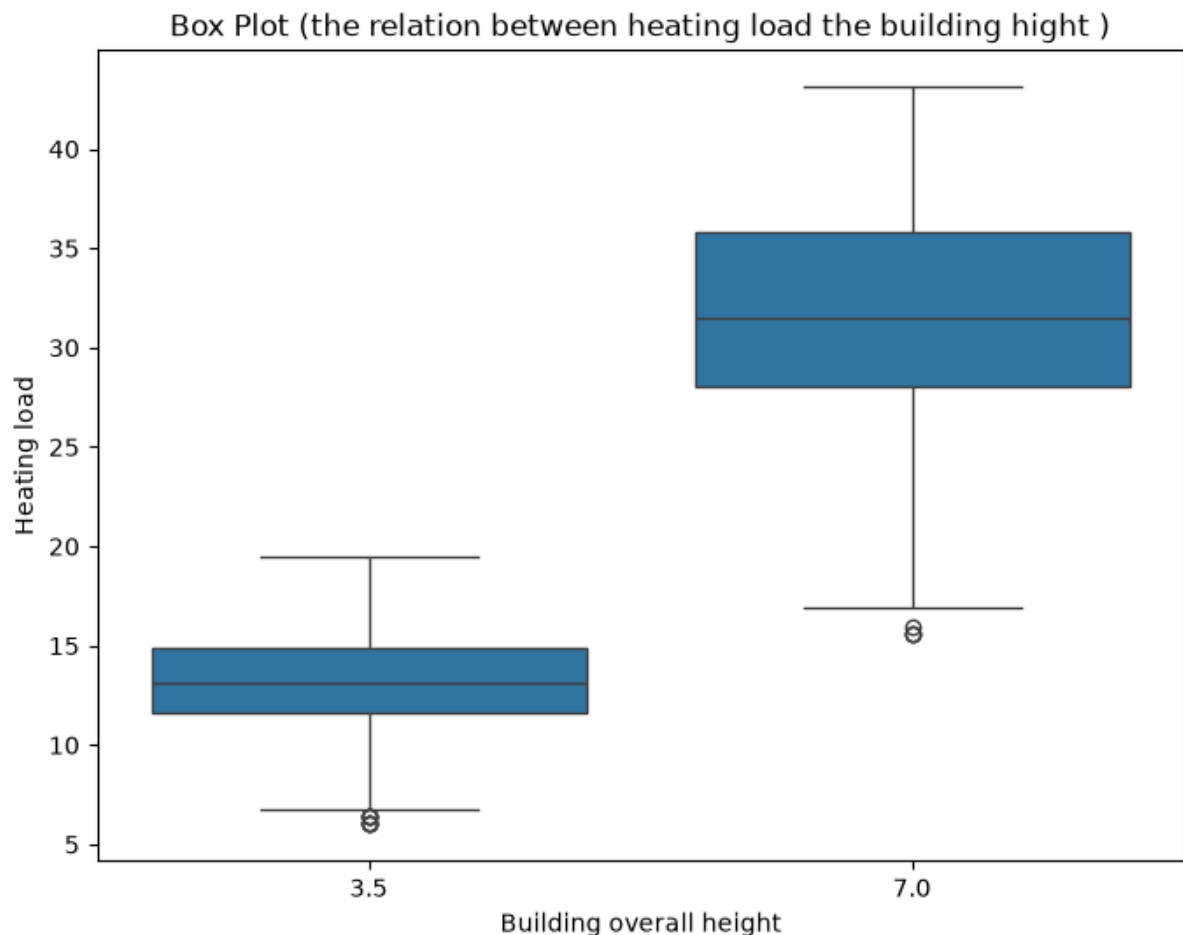
This plot clearly does not follow a normal distribution; instead, it shows a bimodal distribution with two distinct peaks for both heating and cooling loads.

These two peaks indicate that the data is divided into 2 groups, this might happen because of height differences between buildings where taller structures consume significantly higher thermal energy than lower-profile buildings, which will be directly confirmed by the box plot in the subsequent figure.

The Energy Consumption Gap: this bimodal density has two peaks first one with 13.5 mean for spent energy heating load and 16 for the cooling load. While the second peak is 31.8

mean for the heating load and 33 for the cooling load energy spent. The difference between the two peaks exceeds 100% (more than double), demonstrating that these are not random variations but rather indicate the presence of two completely different types of buildings in the sample (low, single-story buildings and tall, multi-story buildings).

3.Box Plot



Description & Observations:

The following box plot shows the heating load distribution based on the building heights in the dataset, the height of building with one story is 3.5m, and the height for building with two floors is 7.0m. The box shows the middle 50% of the data ($IQR = Q_3 - Q_1$), the horizontal line within the box is the median (Q_2), the whiskers extend up to $1.5 \times IQR$ above and under the box, and dots are outliers.

Building with low height 3.5 meters:

- Displays a remarkably low and compact distribution.
- The median heating load nearly 13 kWh/m² (Q_2) while the first quartile (Q_1) is nearly 11 kWh/m² and the third quartile (Q_3) is 15 kWh/m²
- $IQR = 15 - 11 = 4$ kWh/m²
- There are few outlier around 6 to 6.4 kWh/m²

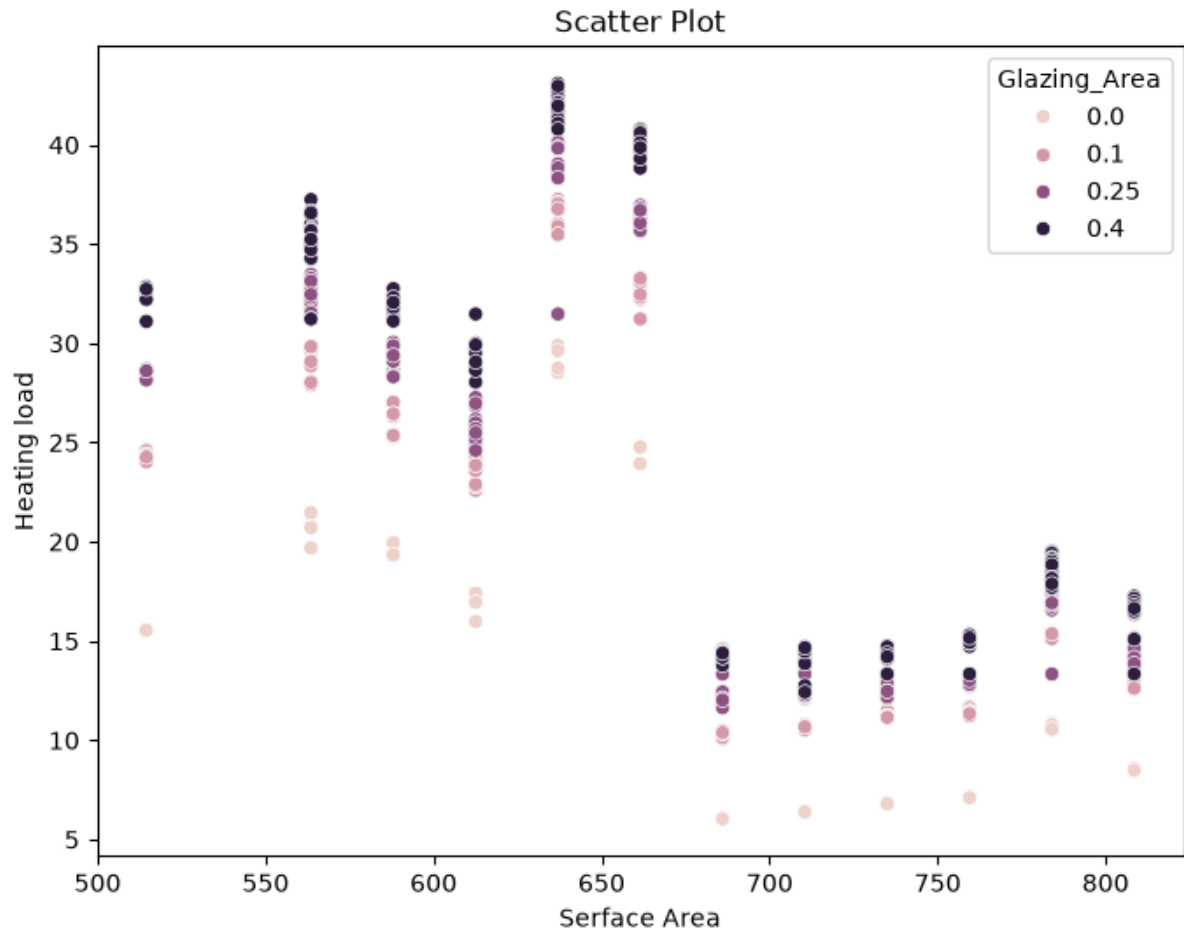
Building with low height 7 meters:

- Displays broader variance
- The median heating load nearly 31 kWh/m² (Q_2) while the first quartile (Q_1) is nearly 28 kWh/m² and the third quartile (Q_3) is 34 kWh/m²
- $IQR = 34 - 28 = 6$ kWh/m²
- There are few outliers around 15 kWh/m²

This visualization empirically validates the hypothesis established during the density distribution analysis: the bimodal peak structure is directly caused by building height. Doubling the building height from 3.5 m to 7.0 m increases the median heating load by more than 130% (from 13 to 31 kWh/m²).

Tall buildings present a larger vertical wall area exposed to colder ambient air and wind, which drastically accelerates conductive heat loss through the building walls and roof area. Also, the wider IQR range ($IQR = 6$ vs. 4 kWh/m²) reveals that taller buildings are far more sensitive to secondary architectural modifications, such as window-to-wall ratios. In conclusion, architectural guidelines and energy codes must impose stricter thermal insulation standards on multi-story buildings to prevent excessive winter thermal dissipation.

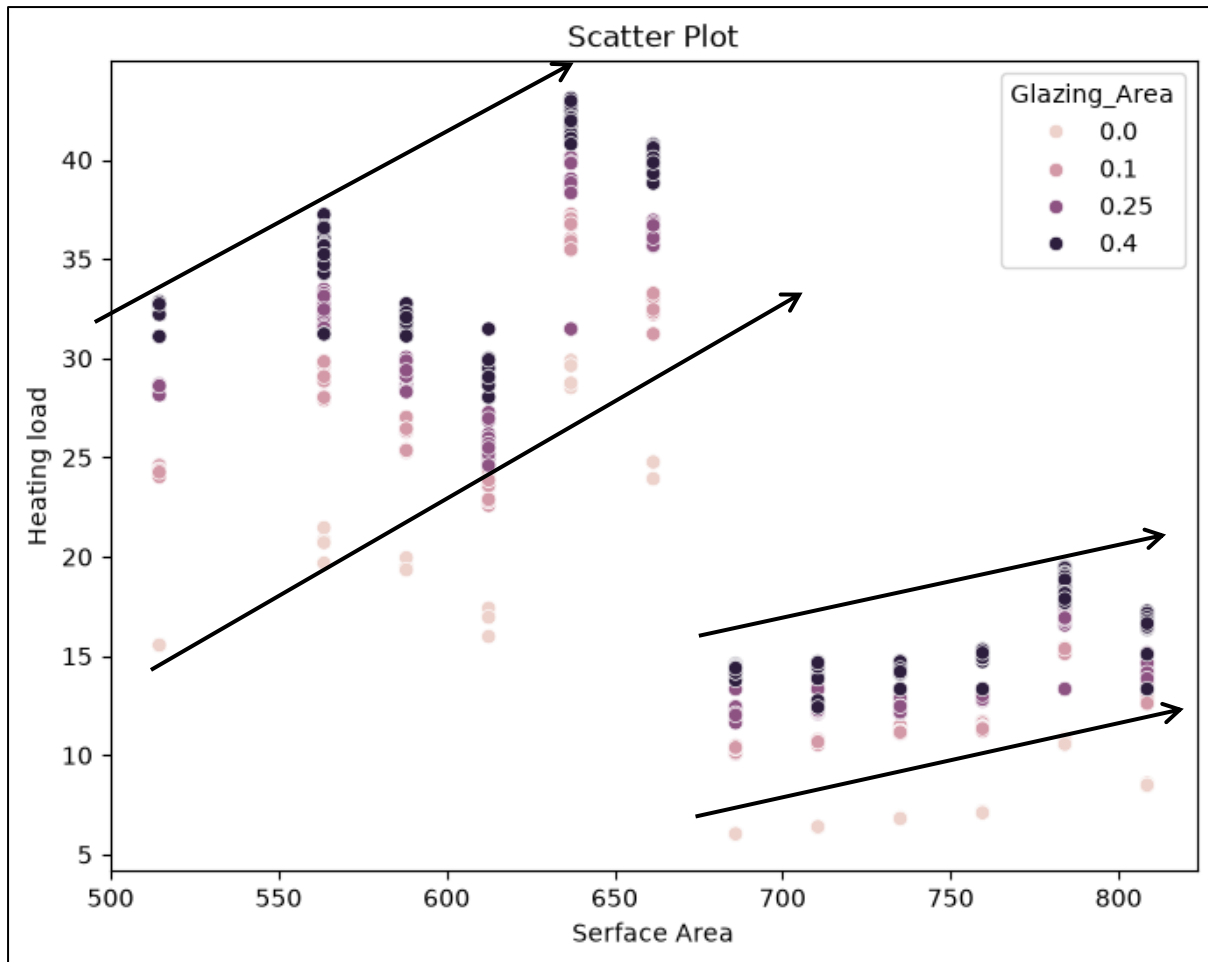
4.Scatter Plot



This Scatter Plot shows the relations between 3 different variables 1. Heating load (Y axis) 2. Surface Area (X axis) 3. Glazing Area(hue).

The dark colors (dark purple) correspond to higher glazing ratios, whereas lighter shades denote minimal or unglazed facades

And as the previous graph the data is divided into two groups according to the building height with a gap in between shows that there are no middle height buildings.



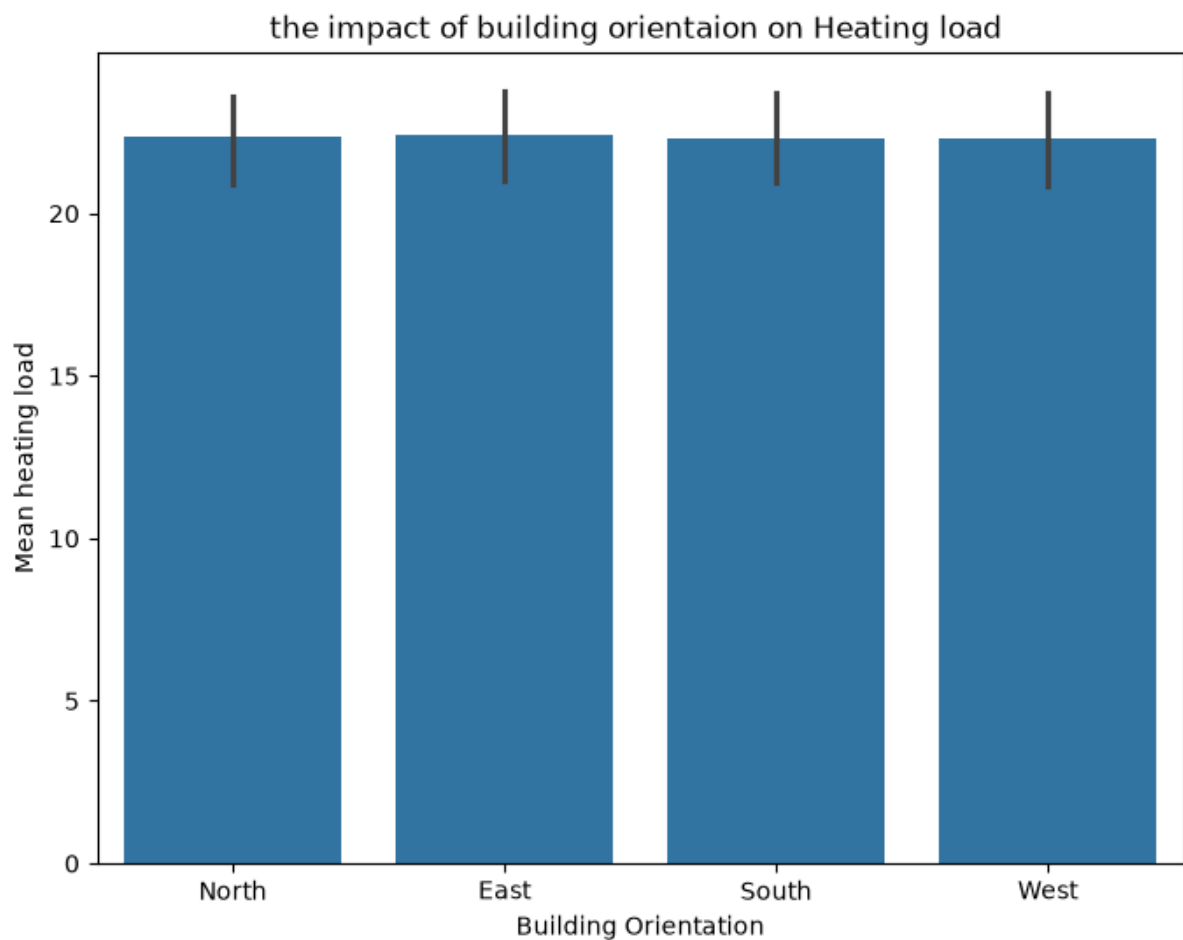
After capturing these two groups of dots between lines the relationship between heating load and surface area is a positive relation, and the sensitivity of heating load to surface area is significantly stronger in multi-story structures than its effect on single-story buildings. (This was inferred from the slope of the points on the curve.)

Also, the top dots in both groups are the dark ones, confirming that glazing area shares a direct positive relationship with heating demand. Building with larger glazing area needs more energy in the heating season.

Overall, the relationship between Surface Area and Heating load is negative (the correlation between them is -0.67). However, inside each group (With height held constant,) increasing the surface area raises the heating load (a direct positive relationship); this is because increased surface area implies a larger wall area exposed to the outside environment, resulting in greater heat loss.

This phenomenon is a classic example of Simpson's Paradox.

5. Bar Plot



In the bar plot figure, the x axis is Building orientation which is the responsible axis to divide the data into groups. The data in this figure has been divided into 4 groups each group represents a certain Orientation (North, South, East, West)

The y axis represents the mean heating load for each Orientation. And the vertical line above each bar represents the error bars, which indicate the 95% confidence interval (uncertainty/variance) around the estimated mean heating load.

This figure shows that the building orientation does not affect the heating load, because all orientation bars have the exact same height.

Each of the four orientations features the exact same variety of buildings (192 buildings identical in terms of glazing area and height); consequently, the building's orientation has no impact on the overall annual average.

4.2 Evaluation

The first step focuses on checking the data accuracy to ensure there are not any missing values or duplicates, because any duplicated records can introduce an artificial data shift and sampling bias. Data accuracy and completeness ensure that differentiation and variance between buildings indicate a real physical factor and not data artifacts.

The relative compactness in physics is volume ratio that can not be out of (0-1) range, so keeping values that equal 100 in the column will create a high leverage outlier shift values to the wrong direction and build a wrong covariance matrix. Capping values greater than 1.0 at 1.0 prevented high-leverage distortion while avoiding record deletion.

Label encoding was used to encode the string values to be numerical, however this type of encoding imposes an artificial ordinal hierarchy on unordered nominal categories. Giving North 0 value and south 2 value makes the North value look less than the South value. However, this didn't make any problem while analysing the data because and as what the heatmap shows the correlation between this feature and other features in the data frame was near to zero.

e-Hot Encoding was avoided because it would create too many extra columns, cluttering the correlation heatmap.

Imputation: Linear Regression vs. Mean vs. Deletion:

A linear regression model was built to predict the missing values in the records.

If deletion was used instead, the data frame will lose more than 40 records, which is a big amount of data to be wasted.

If mean imputation was used, the data shape will change. The density plot proves that the data shape for energy load contains 2 peaks with a gap in between. If mean imputation was implemented, 41 artificial values will be injected into this gap which results to variance flattening shape. It will create a statistical illusion of the existence of medium-sized buildings that do not actually exist within the architectural reality of the sample.

Using linear regression based on structural mass variables enabled the model to predict an energy load appropriate for each building's characteristics (assigning a low value to short buildings and a high value to tall ones). Furthermore, excluding weak variables—such as orientation—prevented the introduction of random noise into the prediction equation.

Many engineers think that directing a building to a certain side is the most appropriate action to decrease the usage of energy load of a building. However, the bar plot shows that there is no relationship between the building orientation and the heating load and the

heatmap shows that there are also no correlations between cooling load and building orientation. This analysis helps in fixing a misunderstanding many engineers have, and it helps to focus on the important features while designing a building.

Simpson's paradox resolution:

The heatmap shows that the correlation between the surface Area and heating load is a negative correlation (-.67) this number may make the engineer thinks that increasing the surface area will decreases energy demand—a completely erroneous conclusion.

The scatter plot and box plot show that there is hidden variable affecting the relation. When height is held constant, the relationship reverses to become a positive direct correlation (+0.45), because increasing the surface area exposed to the outside air inevitably increases heat exchange and heat loss. This conclusion will support architectural policies enforcing volumetric compactness and minimizing excessive facade projections of unjustified.

The Scatter Plot shows the maximum usage energy occur in two story buildings with a big glazing area; the percentage of glazing area increase the heating load 13 kWh/m² for the same building.

increases the heating load for the same building, leading to the following key policy mandates:

1. Establish a mandatory cap on the window-to-wall ratio for multi-story buildings.
2. Mandating high-insulation double or triple glazing (Double/Triple Low-E Glazing) for buildings with open glass facades to compensate for the severe heat leakage identified through analysis.

References

- National Institute of Standards and Technology (NIST) (2012) *NIST/SEMATECH e-Handbook of Statistical Methods*. Gaithersburg: U.S. Department of Commerce. Available at: <https://www.itl.nist.gov/div898/handbook/eda/section3/scatterp.htm>
- Hunter, J.D. (2007) 'Matplotlib: A 2D graphics environment', *Computing in Science & Engineering*, 9(3), pp. 90–95. doi:10.1109/MCSE.2007.55.
- McKinney, W. (2012) *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. Sebastopol: O'Reilly Media.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- Tsanas, A. and Xifara, A. (2012) 'Accurate quantitative estimation of energy performance of residential buildings using statistical machine learning tools', *Energy and Buildings*, 49, pp. 560–567. doi:10.1016/j.enbuild.2012.03.003.
- Waskom, M.L. (2021) 'Seaborn: statistical data visualization', *Journal of Open Source Software*, 6(60), p. 3021. doi:10.21105/joss.03021